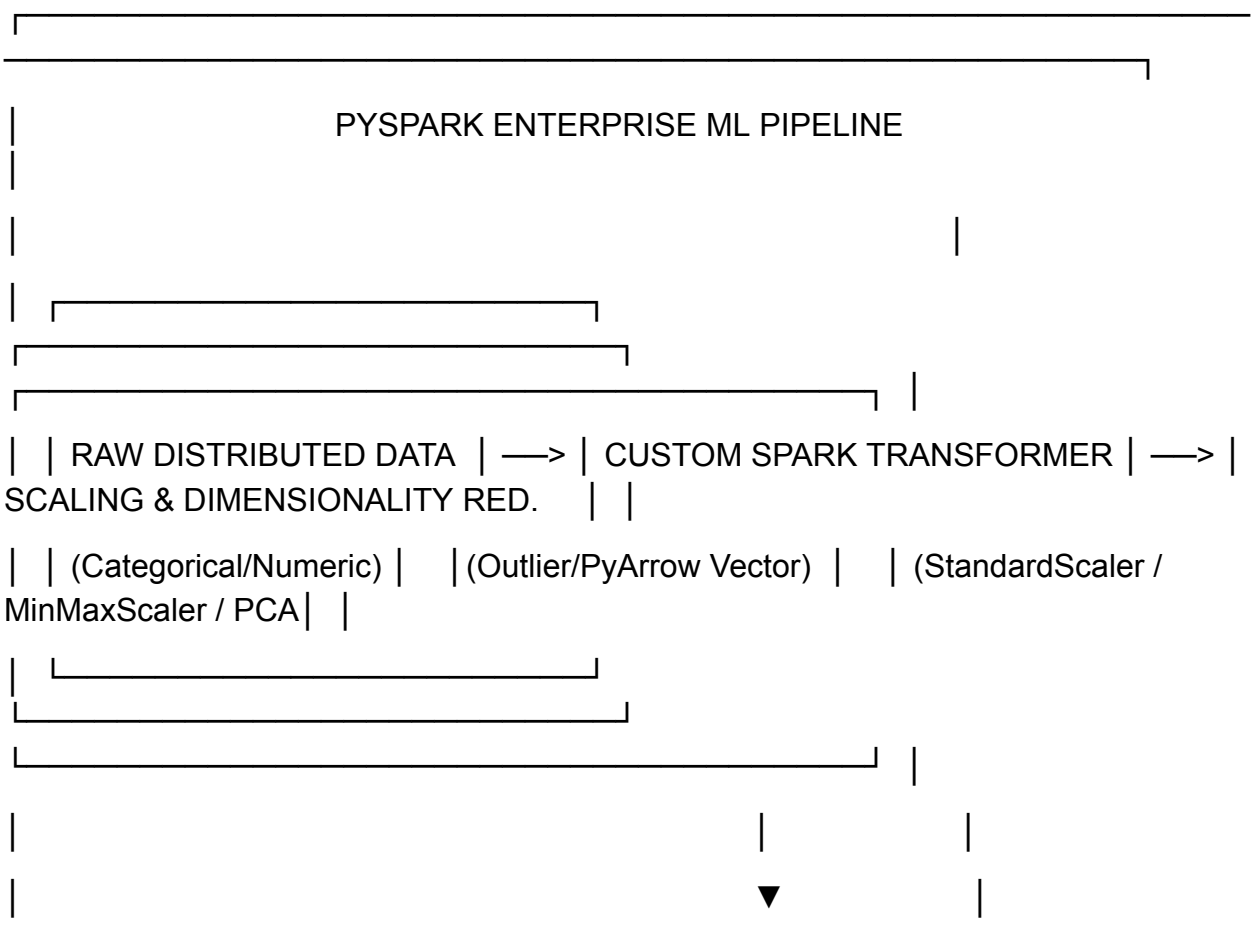
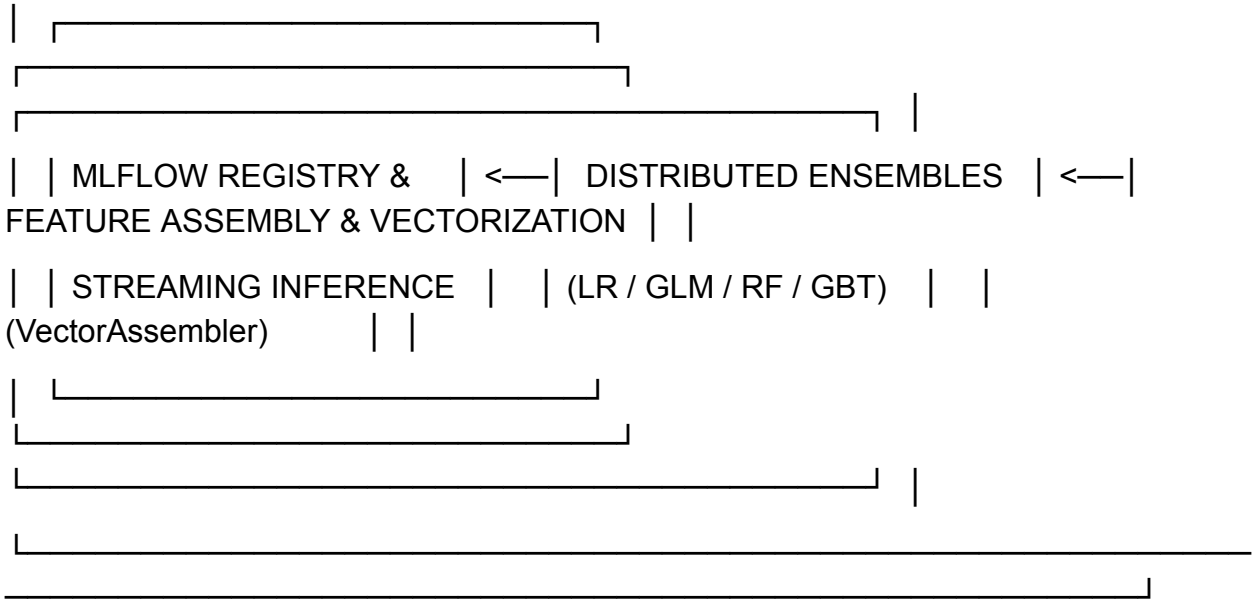
SUBJECT MLLIB: END-TO-END SUPERVISED LEARNING PIPELINES, CUSTOM TRANSFORMERS, AND DISTRIBUTED ENSEMBLES

1. OVERVIEW & OBJECTIVES

Standard machine learning tutorials rely on simple, out-of-the-box estimators on clean data. Production enterprise environments require building **custom, leak-proof PySpark ML Pipelines**, engineering complex feature scale spaces, executing dimensionality reduction via distributed matrix operations, instrumenting experiment tracking with **MLflow**, and scaling ensemble models over massive datasets.

In this advanced assignment, student teams will combine parametric models (Linear Regression, GLM, LinearSVC) and non-parametric tree ensembles (Decision Trees, Random Forests, Gradient-Boosted Trees) into a unified, production-grade PySpark ML pipeline. Teams must implement custom PySpark Transformer classes, write Arrow-accelerated pandas_udf routines, engineer mathematical feature scaling, apply **Distributed PCA**, execute parallel hyperparameter tuning via CrossValidator, and deploy a streaming-compatible model serialization workflow.





2. DATASET REQUIREMENT

Teams must select an enterprise-scale dataset containing continuous, high-cardinality categorical, and noisy temporal columns:

US Flight Delays and Performance Data (USDOT)

- **Dataset Link:** Kaggle - US Flight Delays and Performance Data
- **Task:** Predict arrival delay minutes (Regression) and classify severe delays > 30 mins (Binary Classification).

3. PART A: MATHEMATICAL FOUNDATIONS & ARCHITECTURE REPORT (35 MARKS)

Your team will make a presentation addressing the underlying mathematics and distributed mechanics of PySpark MLlib. Focus on Spark aspect of the subject, for basic models in ML1, no need to repeat the definition, make a quick recap and answer the question directly.

1. Data Scaling Mathematics & Distributed Sparse Operations

- **StandardScaler vs. MinMaxScaler:**
  - Formulate standard z-score scaling: $z = \frac{x-\mu}{\sigma}$
  - Formulate min-max range scaling: $x_{scaled} = \frac{x-x_{min}}{x_{max}-x_{min}} \cdot (max - min) + min$

    - Explain in detail the computational challenge of zero-centering (withMean=True) on sparse distributed vectors (SparseVector) in Spark and how withStd circumvents dense memory expansion.

- **Robust Feature Scaling:** Derive how to handle extreme data skew and outliers using log-transforms ($\ln ln\,(1 + x)$) prior to scaling.

2. Distributed PCA Mechanics

- **Singular Value Decomposition (SVD):** Explain how PySpark computes PCA over distributed matrices using RowMatrix SVD without collecting full covariance matrices to the Driver node.
- **Variance Explained:** Derive the formula for total variance retained across k principal components:

$$Explained\,Variance\,Ratio = \frac{\sum_{i=1}^{k} \lambda_i}{\sum_{j=1}^{p} \lambda_j}$$

3. Parametric Models vs. Tree Ensembles

- **Regression & Classification Formulations:**
    - **ElasticNet OLS & GLM:** Derive the ElasticNet objective function:

$$J(\beta) = \frac{1}{2n}\sum_{i=1}^{n}\left(y_i - x_i^T\beta\right)^2 + \lambda\left[\alpha\|\beta\|_1 + (1 - \alpha)\frac{1}{2}\|\beta\|_2^2\right]$$

Explain link functions $g(E[Y]) = X\beta$ in GLMs (e.g., Gamma/Poisson). Be careful with

    - **LinearSVC:** Derive soft-margin SVM hinge loss:

$$\frac{1}{2}\|w\|^2 + C\sum_{i=1}^{n}\left(0, 1 - y_i\left(w^T x_i + b\right)\right)$$

- **Distributed Ensembles (RF vs. GBT):**
    - **Random Forest:** Explain Bagging, feature subspace sampling ($m = \sqrt{p}$), and parallel tree generation across executors.
    - **Gradient-Boosted Trees:** Explain sequential residual fitting, histogram-based feature binning (maxBins), and why GBTs cannot build trees in parallel.

4. PART B: ADVANCED HANDS-ON IMPLEMENTATION (65 MARKS)

Pipeline Tasks

Task 1: Custom PySpark Transformers & PyArrow Optimization

1. **PyArrow Vectorized UDF:** Write a PySpark pandas_udf (Vectorized UDF using Apache Arrow) to calculate domain-specific continuous metrics (e.g., haversine distance or adjusted depreciation index) with native C-level speed.
2. **OutlierIQRTruncator:** Subclass pyspark.ml.Transformer to compute Q_1, Q_3 IQR bounds across distributed columns and clip extreme values beyond $1.5 \times IQR$.
3. **Leakage-Safe Target Encoding:** Implement target encoding for high-cardinality categorical variables calculated strictly within training splits to prevent data leakage.

Task 2: Pipeline Construction & MLflow Experiment Tracking

Construct an end-to-end Pipeline instrumented with **MLflow**:

- **Preprocessing:** Categorical indexing (StringIndexer), One-Hot Encoding (OneHotEncoder), VectorAssembler, StandardScaler, and PCA(k=10). Plot cumulative explained variance.
- **MLflow Tracking (mlflow.pyspark.ml):**
    - Automatically log pipeline hyperparameters (regParam, maxDepth, numTrees).
    - Log feature importance metrics, explained variance plots, and evaluation artifacts (R^2, RMSE, MAE, AUC-ROC).
    - Register the winning model directly to the **MLflow Model Registry** with stage tags (Staging → Production).

Task 3: The Model Tournament & Distributed Hyperparameter Tuning

Train 5 algorithms under identical train/test splits ($80/20$):

1. **Linear Regression** (ElasticNet $\alpha = 0.5, \lambda = 0.1$)
2. **Generalized Linear Model (GLM)** (Gamma/Poisson family with log link)
3. **LinearSVC** (Binary classification setting)
4. **Random Forest** (RandomForestRegressor / RandomForestClassifier)
5. **Gradient-Boosted Trees** (GBTRegressor / GBTClassifier)

- **Parallel Cross-Validation (CrossValidator):**
    - Configure ParamGridBuilder across tree depths, bin sizes, and regularization grids.
    - Explicitly set CrossValidator(parallelism=4) to execute evaluation trials concurrently across executor slots.

Task 4: Serialization & Real-Time Streaming Inference

- Save the fitted PipelineModel to disk or MLflow artifact storage.
- Write an inference script (inference.py) that loads the serialized model and performs real-time batch predictions on an incoming streaming JSON DataFrame (spark.readStream) to prove zero-downtime schema compatibility and stateless scoring.

5. REQUIRED SUBMISSION DELIVERABLES

Submit your team's project repository containing:

Plaintext

lab-0506-advanced-mllib-teamname/

├── custom_transformers.py # PySpark custom Transformer classes & PyArrow pandas_udf

├── mllib_pipeline.py # Main pipeline script (Ingestion -> Tuning -> MLflow Logging)

├── inference.py # Streaming inference script loading serialized model

├── benchmark_results.py # Metric extraction & Feature Importance plotting script

├── docs/

│ └── REPORT.md # Theoretical derivations, SVD math, and benchmark analysis

├── models/ # Serialized Spark ML Pipeline artifact

└── submit_pipeline.sh # spark-submit production execution script

6. GRADING RUBRIC (TOTAL: 100 MARKS)

Evaluation Category	Criteria & Requirements	Weight / Points
1. Theoretical & Mathematical Rigor	• Deep mathematical breakdown of SVD PCA, ElasticNet OLS, GLM link functions, and Soft-Margin Hinge Loss. • Precise analysis of Bagging vs. Boosting distributed execution.	**35 Marks**
2. Custom Transformers & PyArrow	• Functional implementation of custom PySpark Transformer subclasses (OutlierIQRTruncator) operating natively on DataFrames. • Native execution of Arrow-based pandas_udf within feature processing steps.	**15 Marks**
3. ML Pipeline & PCA	• End-to-end Pipeline integrating categorical encoding, custom scaling, VectorAssembler, and PCA. • Correct evaluation of explained variance ratios.	**15 Marks**
4. Multi-Model Tuning & Parallel CV	• Training of LR, GLM, LinearSVC, RF, and GBT.	**20 Marks**

Evaluation Category	Criteria & Requirements	Weight / Points
	• Parallel execution of 5-fold CrossValidator with ParamGridBuilder and parallelism tuning.	
5. MLflow Instrumentation & Streaming	• Successful logging of metrics, hyperparameter grid runs, and MLflow model registry artifacts. • Working pipeline serialization and live streaming inference demonstration via spark.readStream.	